The Testing Process

1. Overview

This report documents the testing process for the Scientific Calculator application developed for SE 317 Lab 7. The application follows the Model-View-Controller (MVC) pattern and includes features such as basic arithmetic, advanced math functions, memory operations, and a GUI.

Testing was conducted at two levels:

- **Model Testing** using JUnit
- **GUI Testing** using `SwingUtilities` to simulate user interactions

The purpose of this report is to verify that the calculator:

- Computes correct results for all supported operations
- Handles edge cases (e.g., divide by zero, negative inputs for square root)
- Displays correct and expected values
- Visually responds to user interactions

---

2. Testing Setup

Tools Used:

- **JUnit 4**: For unit tests of `CalculatorModel`
- **Java SwingUtilities**: For GUI simulation via programmatic button clicks
- **Reflection**: Used in GUI tests to access private components (e.g., button arrays)

System Under Test:

- `CalculatorModel`: Provides all core math and memory logic
- `CalculatorView`: Contains GUI components (buttons, display)
- `CalculatorController`: Binds the view to the model

All components were tested together.

---

3. Testing Categories & Results

A. Model-Level Unit Tests

Tests covered:

- Basic operations: addition, subtraction, multiplication, division
- Decimal and negative value behavior
- Advanced functions: square and square root
- Memory operations: add, subtract, clear, recall
- Edge cases: divide by zero, square root of a negative number

Sample test coverage:

- `testAddition()`: verifies `2 + 3 = 5.0`
- `testMemoryClear()`: ensures `MC` resets memory to `0.0`
- `testDivideByZero()`: division by zero returns `"Infinity"` (mirrors Java behavior)

➡ **Result:** All unit tests passed, verifying that the model handles both normal and edge-case inputs correctly. Proof to come later in the report.

B. GUI-Level Functional Tests

Simulated user interactions were used to test:

- Input sequences and display updates
- Operation result correctness via button clicks
- Error handling (e.g., dividing by 0)
- Complex interaction (e.g., `3 + 3 =` → `M+` → `MR`)
- Display formatting (only operands/results shown — no symbols)

Sample test cases:

- `testAddition()`: clicks 2 → `+` → 3 → `=`, expects `5`
- `testInvalidSquareRoot()`: negates 4 → √, expects `Error`
- `testMemoryRecallAfterAddition()`: performs M+ then MR, confirms result
- `noOperationSymbolDisplayedWhilePerformingDivisionOperation()`: confirms UI doesn't show `/` during calculation

➡ **Result:** All interactions behaved correctly. Results were displayed as expected and no operator symbols leaked to the display.

C. Visual Feedback Testing

To validate user experience, tests ensured:

- Pressed operation buttons change the background to gray
- Buttons retain their visual state until `=` is pressed.
- Buttons revert to their original styling after the operation completes

Sample visual test cases:

- `testAdditionButtonGrayedOutWhenOperating()`: verifies background change on `+` press
- `testSubtractionButtonGrayedOutWhenOperating()`: same for `-`
- All tests compare actual button background colors using `getBackground().toString()`

➡ **Result:** Buttons reflected correct UI state changes during operations.

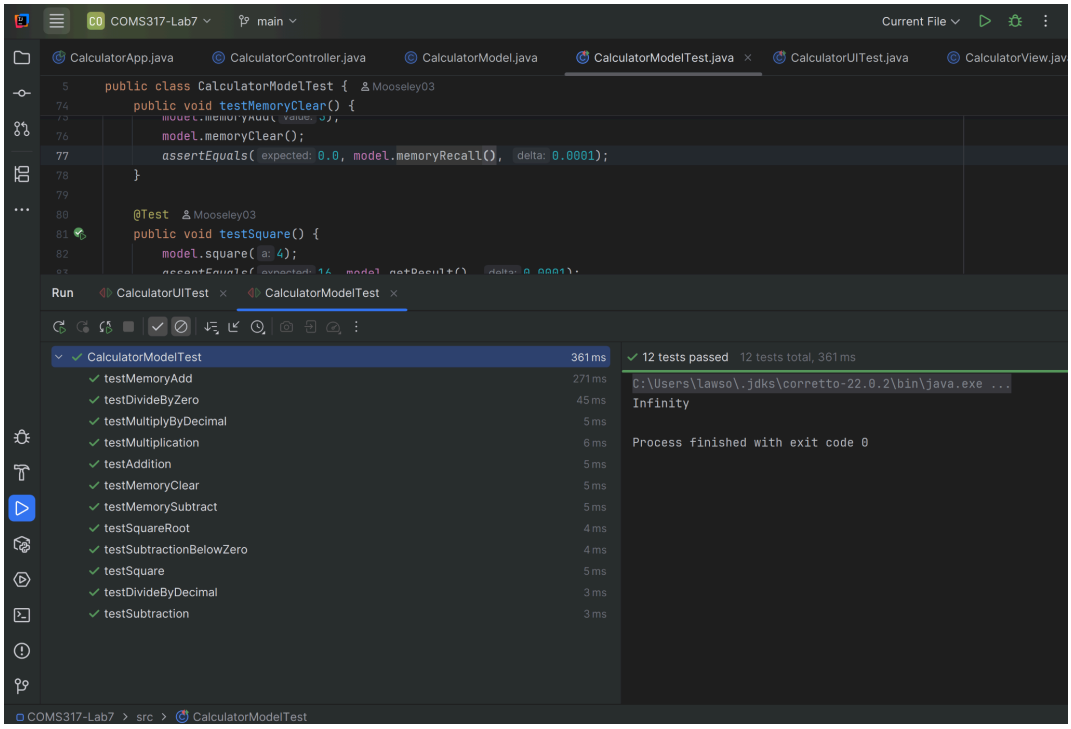
---

4. Summary of Results

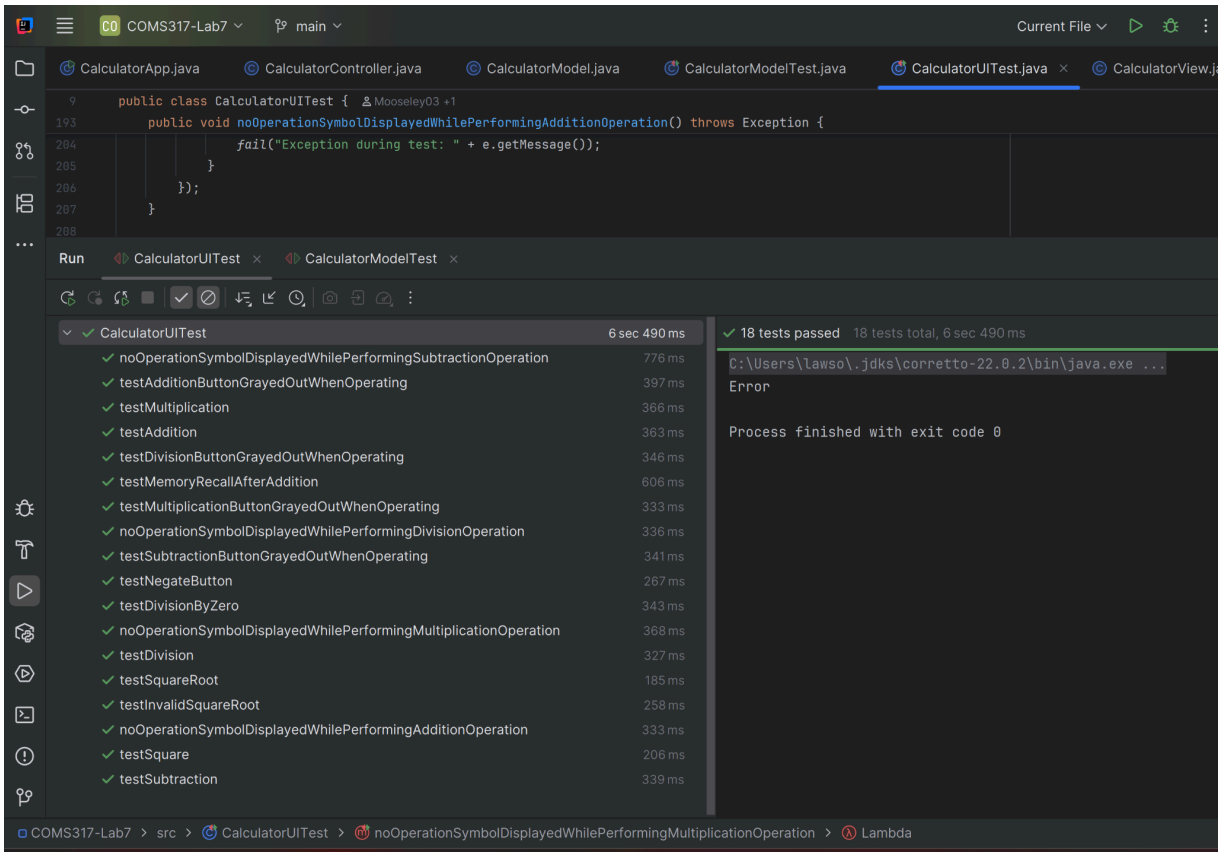
Test Area	Status	Notes
Model functionality	✅ Pass	All operations accurate, including edge cases
GUI behavior	✅ Pass	Input → display interaction correct and smooth
Display formatting	✅ Pass	Only operands/results shown — no operation symbols
Button visual states	✅ Pass	Buttons visually respond and reset properly after execution

---

5. Proof Of Results



Model Test Results



GUI Test Results

---

6. Conclusion

The calculator application meets all specified requirements for correctness, user interaction, and visual behavior. Testing confirmed both logic integrity in the model and interactive correctness in the UI. No bugs, regressions, or visual glitches were found during testing.

This concludes the documented test process and serves as evidence of reliable behavior across core functionality, edge cases, and user-facing responsiveness.